Projet Sens-Ex : Tutoriel pour le traitement des données multimodales CMC

Rédaction : Sangwan Jeon, Ariane Robert, Luisa Troncone
Compilation et traduction : Emili Avramovic

Ce premier tutoriel a pour objectif d'expliciter la méthode de collecte des données selon les différentes plateformes sources des données : Twitch, TikTok, YouTube et X (Twitter).

Deux méthodes complémentaires sont à prévoir en fonction des plateformes, des types de données et des stades du processus de traitement :

- Utilisation des pipelines intégrant le téléchargement des données, l'extraction fondée sur les time codes, la transcription automatique pour les données sur YouTube, TikTok et Twitch.
- Utilisation de freewares pour l'édition (extraction des segments qui nous intéressent) et la transcription automatique des fichiers vidéo déjà téléchargés, ainsi que pour l'enregistrement d'écran pour les données sur Twitter/X.

1 TELECHARGEMENT DES DONNEES YOUTUBE A L'AIDE D'UN PIPELINE – L. TRONCONE 2

2 TELECHARGER DES DONNEES TIKTOK A L'AIDE D'UN PIPELINE – L. TRONCONCE 7

3 TELECHARGER ET ALIGNER DES DONNEES TWITCH A L'AIDE D'UN PIPELINE – A. ROBERT 11

3.1 INSTALLATION DES REFERENTIELS 11
3.2 TELECHARGEMENT DES DONNEES DE TWITCH 12
3.3 TRANSCRIPTION DES DONNEES AUDIO 14
3.4 ALIGNEMENT DU CHAT ET DE LA TRANSCRIPTION AUDIO 15
3.5 QUELQUES OUTILS ADDITIONNELS 17

4 TELECHARGER DES DONNEES TWITTER/X A L'AIDE DES FREEWARES – S. JEON 20

4.1 ÉDITER UNE VIDEO AVEC VIDEOPROC VLOGGER 20
4.2 ENREGISTRER L'ECRAN AVEC OBS STUDIO 20
4.3 TRANSCRIRE AUTOMATIQUEMENT UN FICHIER AUDIO AVEC SPEECHMATICS 21
4.4 TRANSCRIRE AUTOMATIQUEMENT UN FICHIER AUDIO AVEC ASR_BATCH.PY 22

1 Téléchargement des données YouTube à l'aide d'un pipeline – L. Troncone

Les données YouTube sont traitées à l'aide du code disponible au lien suivant : https://colab.research.google.com/drive/11K3lSMbrSceYztb3z1mM-PSpUu_h7BE_?usp=sharing

Ce pipeline accessible sur Google CoLab permet l'obtention des fichiers suivants :

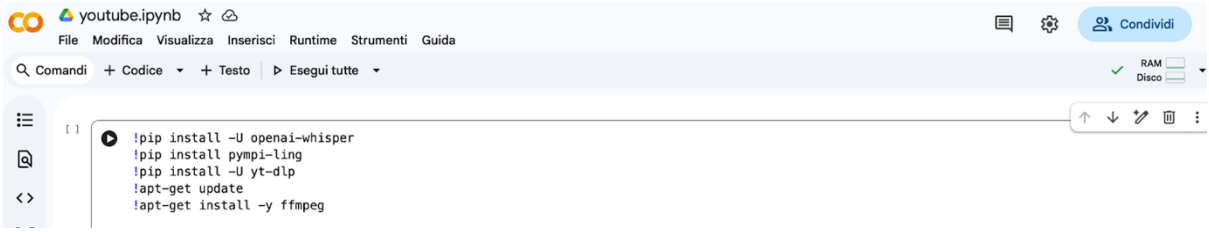
- La vidéo entière (.mp4)
- Des segments choisis (.mp4)
- Leur audio (.wav)
- Leurs transcriptions respectives (.eaf compatible avec ELAN)

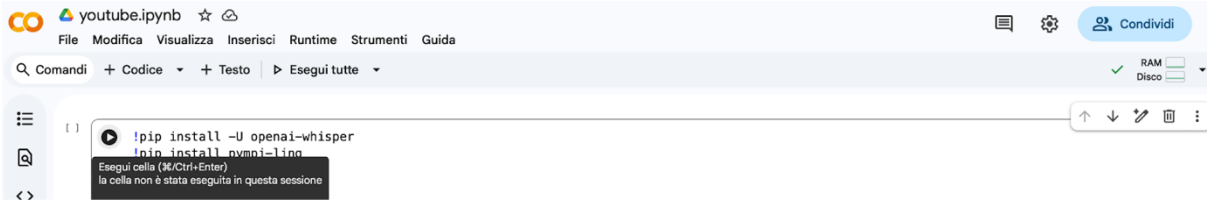
À partir des informations suivantes :

- L'URL de la vidéo YouTube
- Le temps de début du segment (timecode en seconde)
- Le temps de fin du segment (timecode en seconde)

Ci-après, le processus exemplifié à l'aide de captures d'écran :

1) Exécuter la première cellule pour installer les packages nécessaires à la session à l'aide du bouton play.

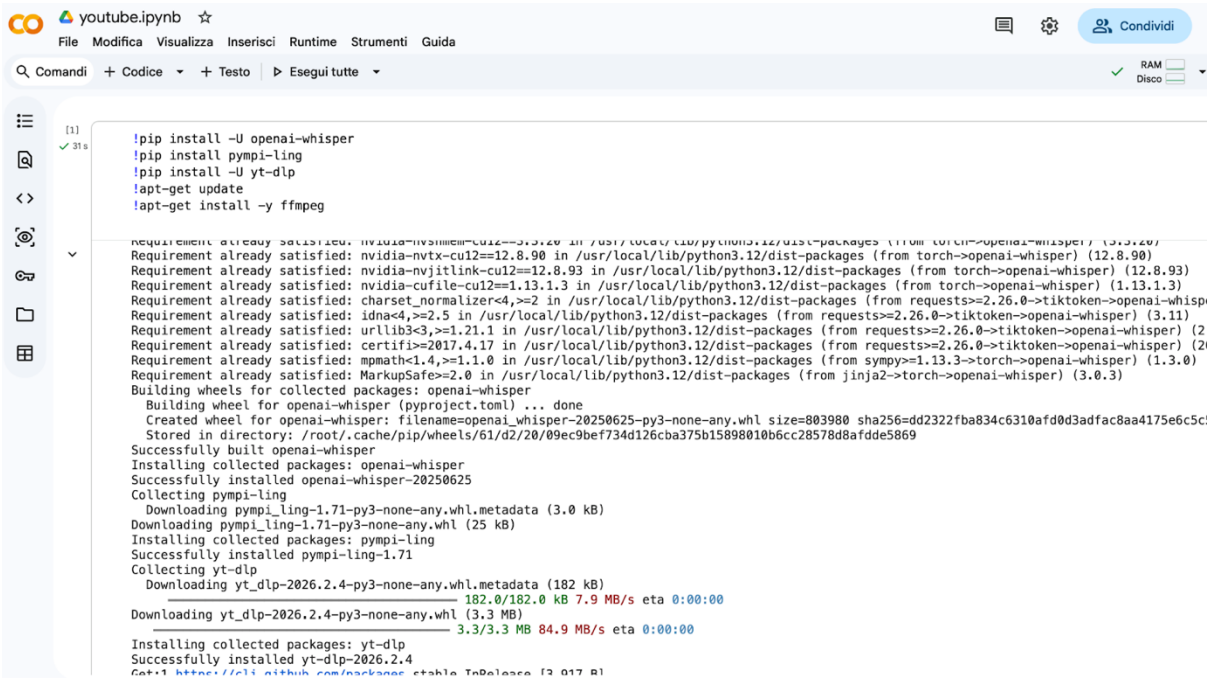




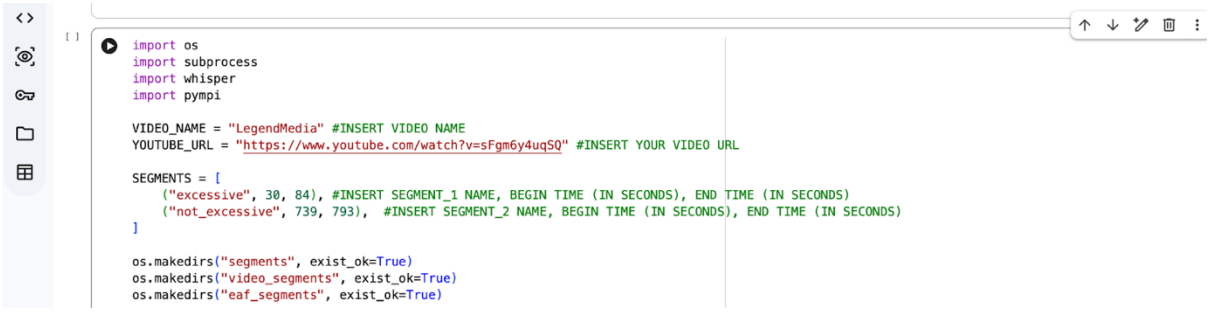
1) Cellule en cours d'exécution.



2) Cellule exécutée reconnaissable au symbole vert de validation.



3) Dans la cellule suivante, il faut insérer un nom pour la vidéo, son URL, le nom et les timecodes des segments choisis.
   Dans cette image, le nom de la vidéo choisie est *LegendMedia*, et l'URL est https://www.youtube.com/watch?v=sFgm6y4uqSQ.
   Les noms des deux segments sont *excessive* et *not_excessive*, et leurs timecodes sont respectivement 30-84 s et 739-793 s.
   Nb : nous rappelons que la conversion des heures vers les secondes est une multiplication x3600 et des minutes vers les secondes une multiplication x60 (par exemple 02 : 37 : 44 = 2*3600 + 37*60 + 44 = 9464).



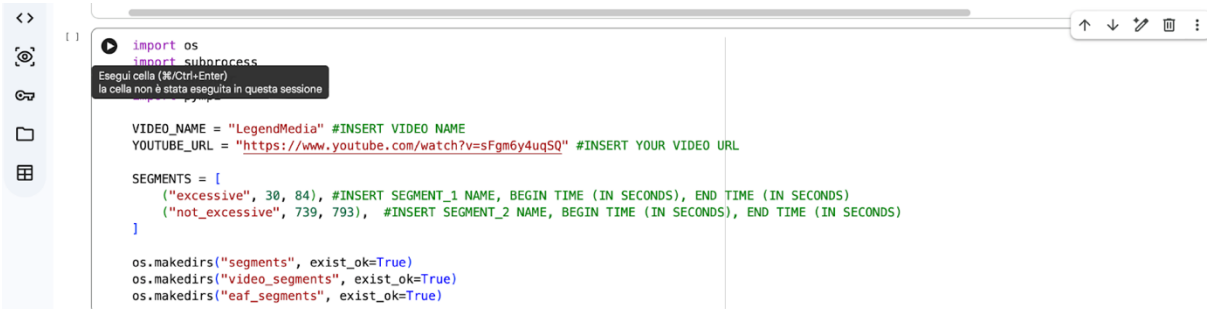
```
import os
import subprocess
import whisper
import pympi

VIDEO_NAME = "LegendMedia" #INSERT VIDEO NAME
YOUTUBE_URL = "https://www.youtube.com/watch?v=sFgm6y4uqSQ" #INSERT YOUR VIDEO URL

SEGMENTS = [
    ("excessive", 30, 84), #INSERT SEGMENT_1 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
    ("not_excessive", 739, 793),  #INSERT SEGMENT_2 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
]

os.makedirs("segments", exist_ok=True)
os.makedirs("video_segments", exist_ok=True)
os.makedirs("eaf_segments", exist_ok=True)
```

4) Exécuter cette cellule.



```
import os
import subprocess
```

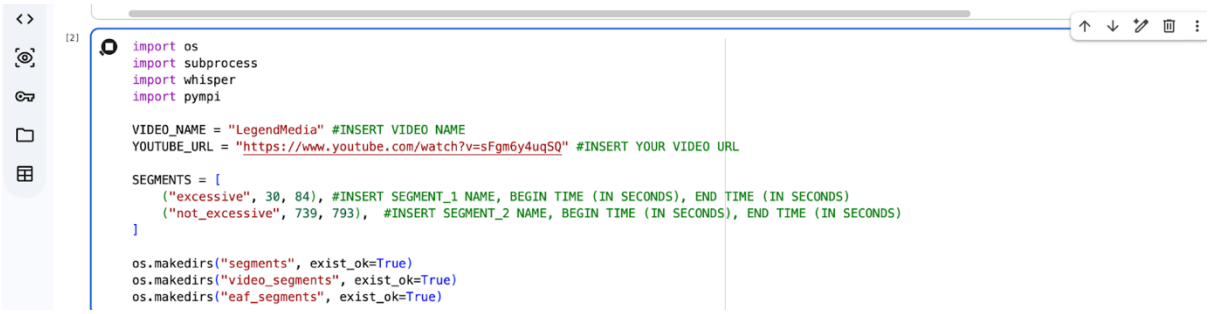
Esegui cella (⌘/Ctrl+Enter)
la cella non è stata eseguita in questa sessione

```
VIDEO_NAME = "LegendMedia" #INSERT VIDEO NAME
YOUTUBE_URL = "https://www.youtube.com/watch?v=sFgm6y4uqSQ" #INSERT YOUR VIDEO URL

SEGMENTS = [
    ("excessive", 30, 84), #INSERT SEGMENT_1 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
    ("not_excessive", 739, 793),  #INSERT SEGMENT_2 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
]

os.makedirs("segments", exist_ok=True)
os.makedirs("video_segments", exist_ok=True)
os.makedirs("eaf_segments", exist_ok=True)
```

5) Ces cellules permettent de définir l'ensemble des variables.



```
import os
import subprocess
import whisper
import pympi

VIDEO_NAME = "LegendMedia" #INSERT VIDEO NAME
YOUTUBE_URL = "https://www.youtube.com/watch?v=sFgm6y4uqSQ" #INSERT YOUR VIDEO URL

SEGMENTS = [
    ("excessive", 30, 84), #INSERT SEGMENT_1 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
    ("not_excessive", 739, 793),  #INSERT SEGMENT_2 NAME, BEGIN TIME (IN SECONDS), END TIME (IN SECONDS)
]

os.makedirs("segments", exist_ok=True)
os.makedirs("video_segments", exist_ok=True)
os.makedirs("eaf_segments", exist_ok=True)
```

6) Nous pouvons maintenant passer au traitement des vidéos. Cette cellule permet de télécharger la vidéo que vous avez sélectionnée.



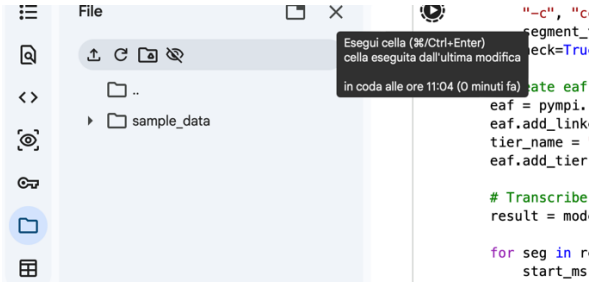
```
# 1. Download entire video
video_file = f"{VIDEO_NAME}.mp4"
subprocess.run([
    "yt-dlp",
    "-f", "mp4",
    "-o", video_file,
    YOUTUBE_URL
], check=True)
```

7) Une fois la vidéo téléchargée, elle est retrouvable dans cet onglet à gauche de l'écran :



```
    "-o", video_file,
    YOUTUBE_URL
], check=True)
```

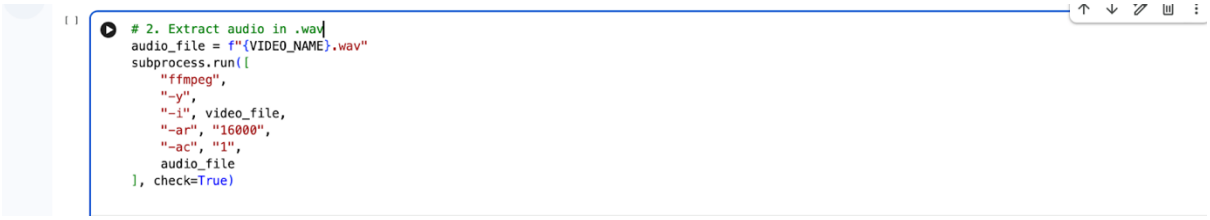
8) Après avoir cliqué sur le dossier, vous pouvez retrouver cet environnement où vous pourrez retrouver tous vos fichiers.



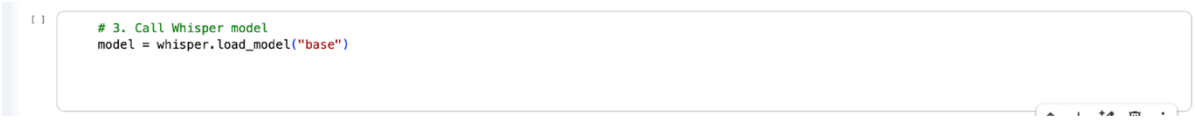
9) Exécuter le code de téléchargement .



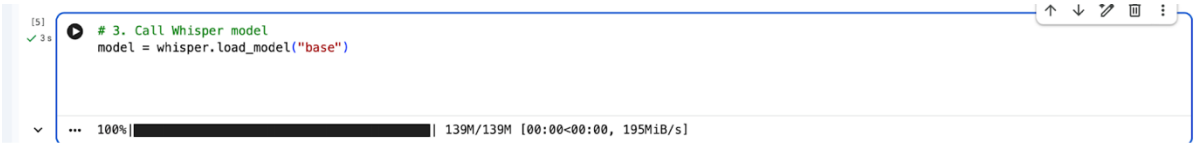
10) La prochaine étape est l'extraction de l'audio au format .wav, la transcription sera générée à partir de ce fichier – exécuter cette cellule.



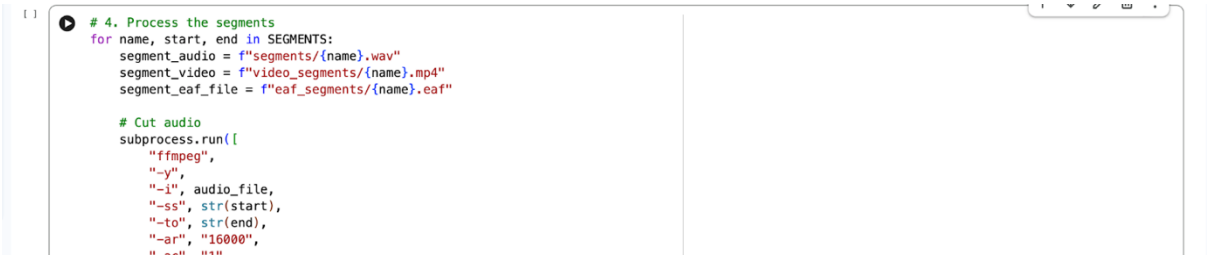
11) Importation du modèle Whisper. C'est un logiciel développé par OpenAI que nous utilisons pour obtenir une transcription automatique de l'audio – exécuter cette cellule.

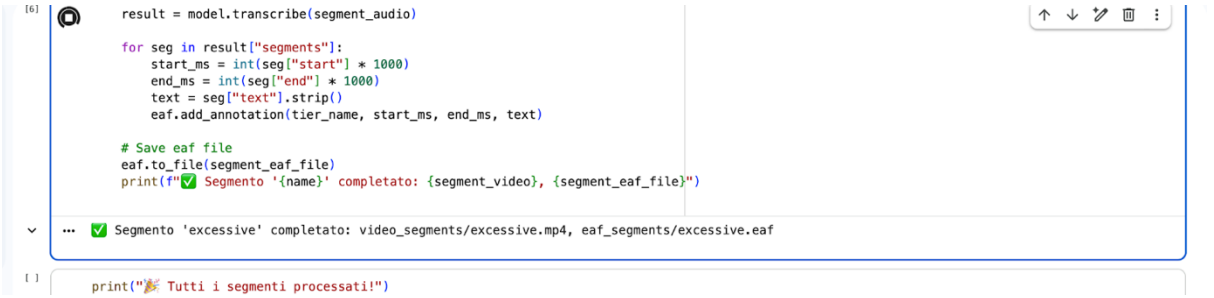


12) Une fois l'importation terminée, vous devriez voir une barre de progression noire remplie à 100%.

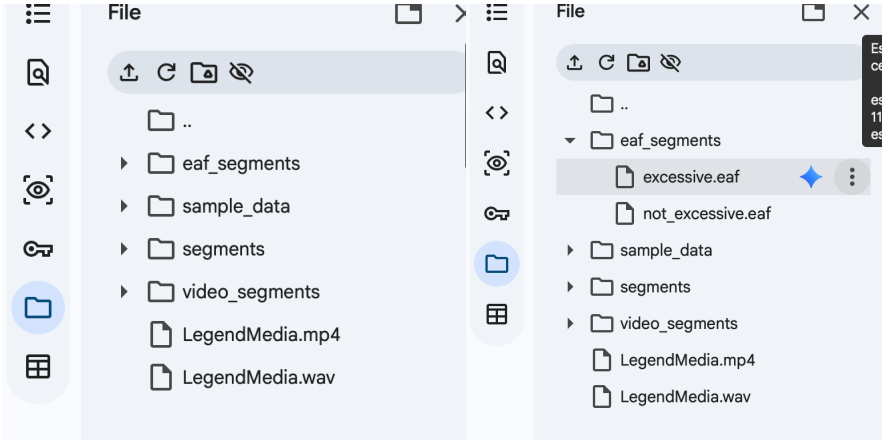


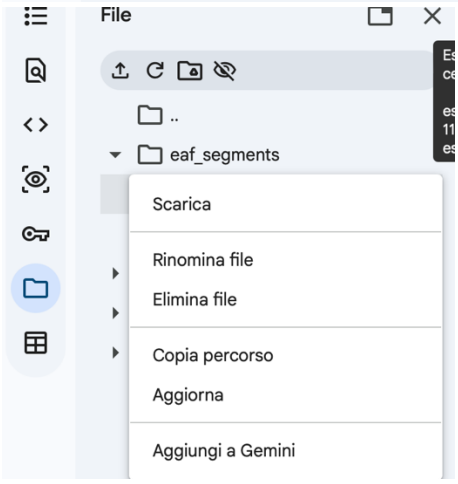
13) La prochaine étape est le traitement des différents segments : découpage de l'audio et transcription de l'ensemble des fichiers multimédia.





14) Enfin, vous pouvez accéder à l'ensemble des fichiers produits dans la liste à gauche que vous pouvez télécharger à l'aide des trois points.





ici *scarica* pour téléchargement

2 Téléchargement des données TikTok à l'aide d'un pipeline – L. Tronconce

Les données TikTok peuvent être traitées à l'aide du lien Google Collab suivant : https://colab.research.google.com/drive/1zjfH7vtEWzaPjVSAaVnIwRfqjlEPblGJ?usp=sharing
Il est toutefois important de respecter les bonnes pratiques d'un traitement éthique des données dans le cadre de la recherche, en particulier lorsqu'il s'agit de données personnelles (et potentiellement sensibles), comme les données issues des réseaux sociaux. Il est recommandé de demander un accès à l'API de recherche de TikTok : https://developers.tiktok.com/products/research-api/.
Pour des raisons de conformité, voir également : https://www.tiktok.com/legal/page/global/terms-of-service-research-api/en.

Ce pipeline accessible sur Google CoLab permet l'obtention des fichiers suivants :

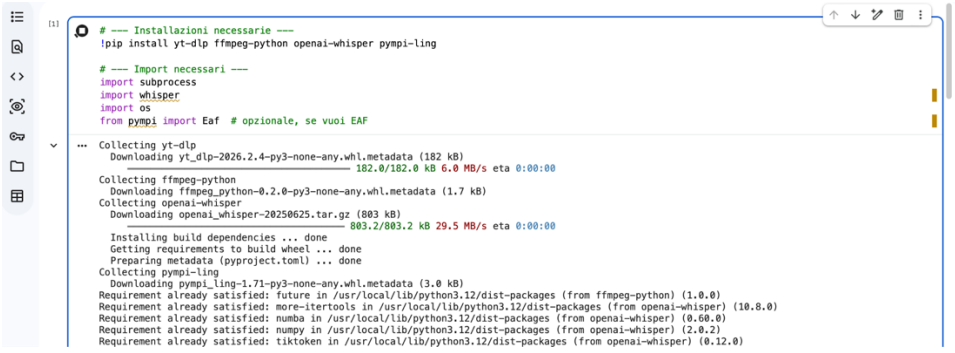
- La vidéo entière (.mp4)
- Leur audio (.wav)
- Leurs transcriptions respectives (.eaf compatible avec ELAN, et .txt)

Ci-après, le processus exemplifié à l'aide de captures d'écran :

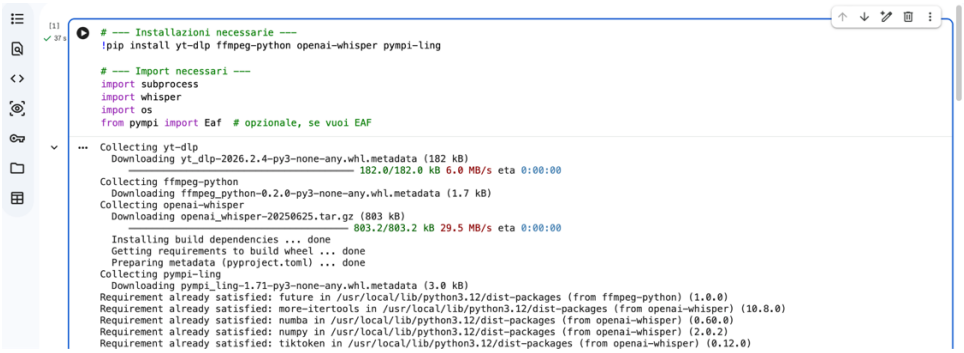
1) Exécuter la première cellule pour installer les packages nécessaires à la session à l'aide du bouton de lecture.



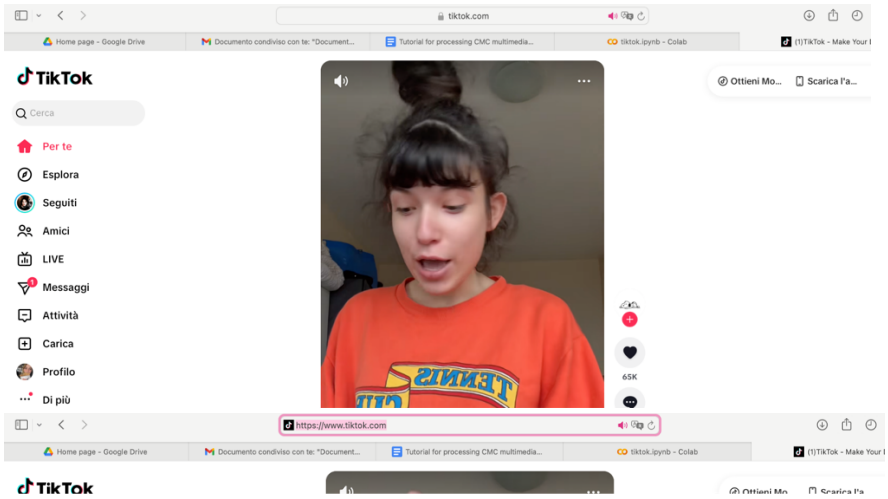
2) Cellule en cours d'exécution.



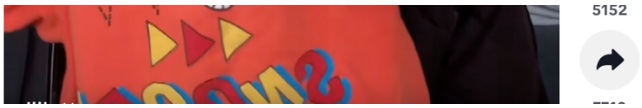
3) Cellule exécutée reconnaissable au symbole vert de validation.



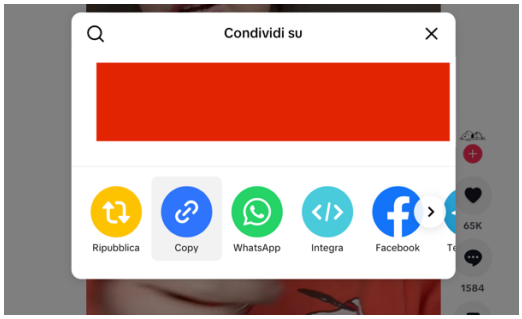
4) Avant de continuer, il faut trouver le lien URL du TikTok choisi, qui ne se trouve pas dans la barre du haut.



5) Pour y accéder, il faut cliquer sur le bouton partager (*share*) en bas à droite de la vidéo.



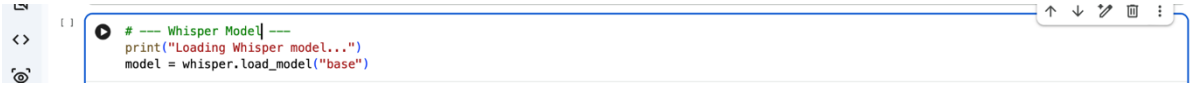
6) Une fois cette page pop-up ouverte il faut cliquer sur copier (*copy*) pour obtenir le lien voulu.



7) Il faut retourner sur CoLab pour coller le lien obtenu et exécuter la cellule.



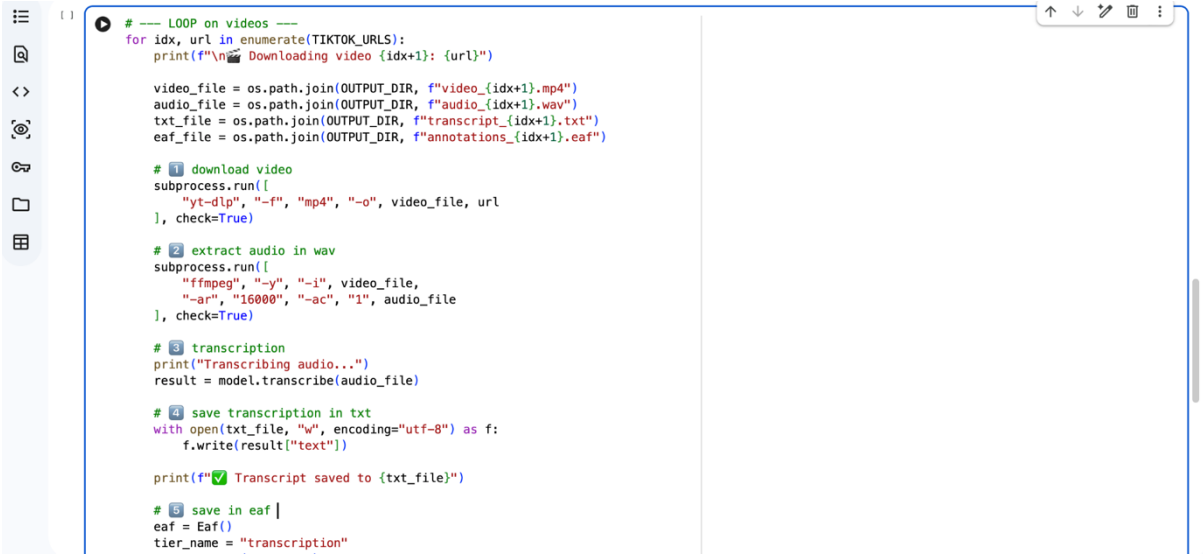
8) Importation du modèle Whisper. C'est un logiciel développé par OpenAI que nous utilisons pour obtenir une transcription automatique de l'audio – exécuter cette cellule.



9) Une fois l'importation terminée, vous devriez voir une barre de progression noire remplie à 100%.



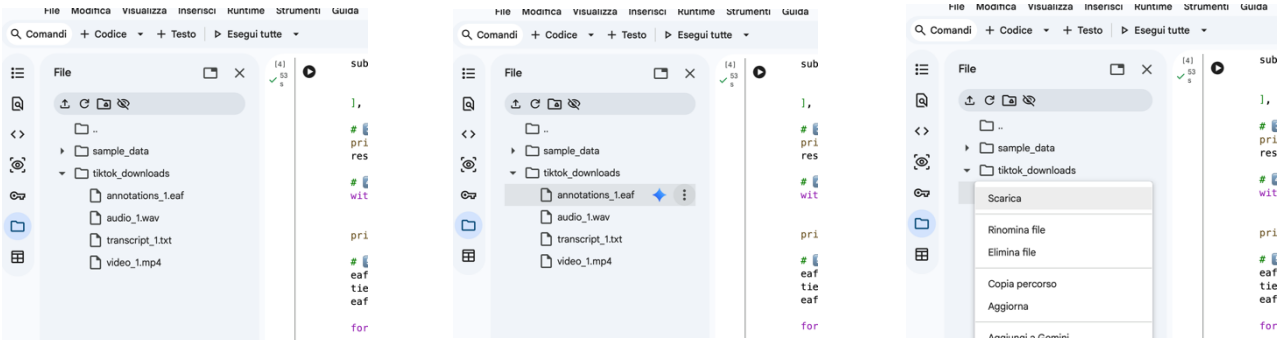
10) Nous pouvons maintenant passer au traitement de la vidéo. Exécuter cette cellule.



11) Pour accéder à l'ensemble des fichiers générés, regardez à gauche de l'écran : une icône de dossier s'y trouve.



12) Enfin, vous pouvez accéder à l'ensemble des fichiers produits dans le sous-dossier *tiktok_downloads* que vous pouvez télécharger à l'aide des trois points.



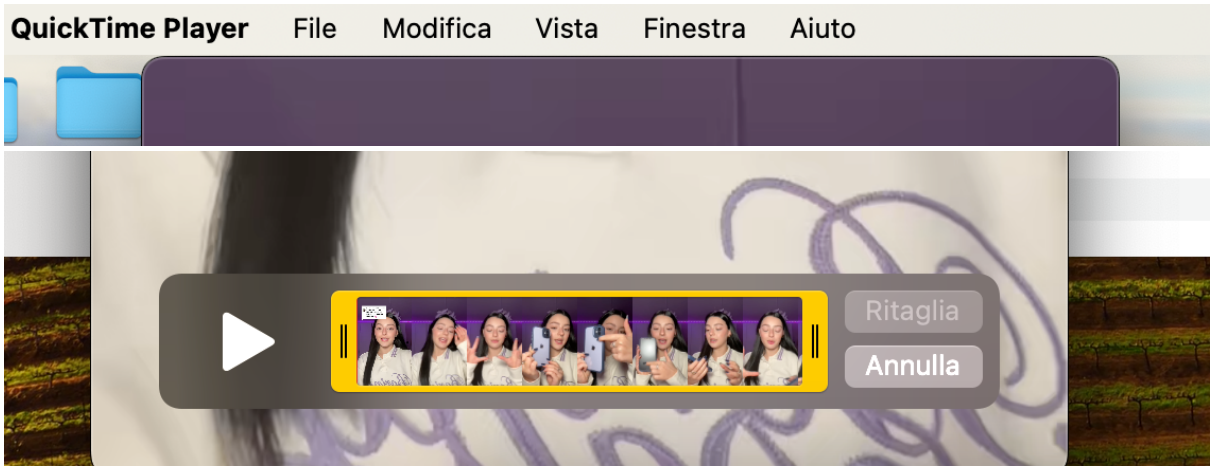
Ici *scarica* pour téléchargement.

Pour chaque nouvelle vidéo TikTok à traiter, veillez à supprimer les données précédemment téléchargées.

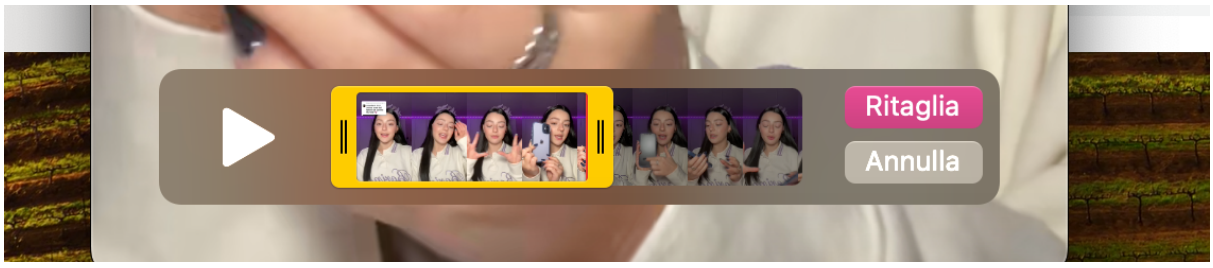
13) Si la vidéo est trop longue, vous devrez la découper à l'aide d'un logiciel de montage vidéo, par exemple :

- Sous Windows : l'éditeur de photos (versions récentes) ou Windows Movie Maker (versions antérieures)
- Sous macOS : le lecteur vidéo (voir illustration).

14) Sur QuickTime Player (MacOS), il faut sélectionner l'onglet *Modifier* (ici *Modifica*) et choisir l'option *couper* qui doit vous donner cette vue :



15) Sélectionner le segment voulu et appuyer sur le bouton couper (ici *Ritaglia*)

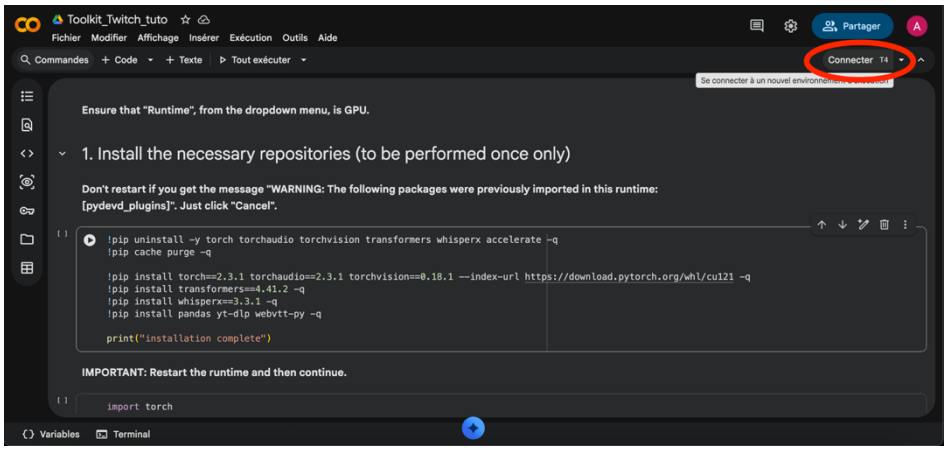


3 Téléchargement et alignement des données Twitch à l'aide d'un pipeline – A. Robert

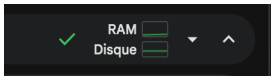
Le toolkit a été développé par S. Coats (2024) et adapté par A. Robert (2025). Il doit être exécuté dans le Google Collab disponible ici. Il comprends plusieurs étapes :

- L'installation de référentiels
- Le téléchargement des données de Twitch
- La transcription des données audio
- L'alignement du chat et de la transcription audio
- Quelques outils additionnels

Avant de commencer à exécuter les différentes cellules du script, veillez à vous **connecter au GPU** à l'aide du bouton en haut à droite :



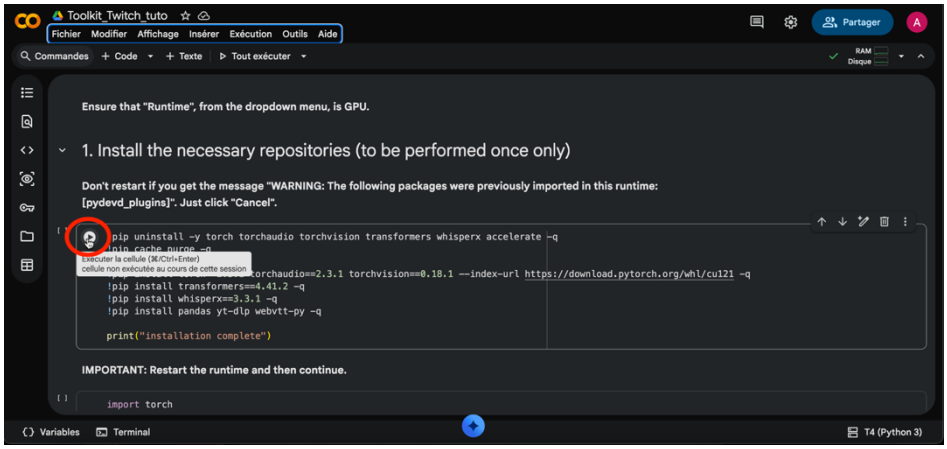
Le bouton devrait alors apparaître comme ceci :



Important : lors de l'exécution des différentes cellules, une fenêtre s'affichera peut-être pour vous demander de redémarrer votre session. Il est important de **ne pas la redémarrer sauf lorsque cela est indiqué dans le tutoriel**.

3.1 Installation des référentiels

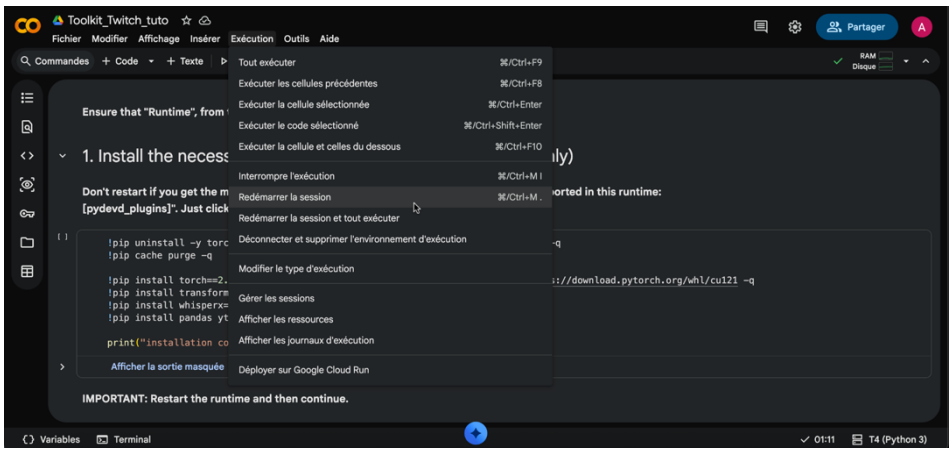
ÉTAPE 1 : lancer la première cellule à l'aide du bouton d'exécution comme sur l'image :



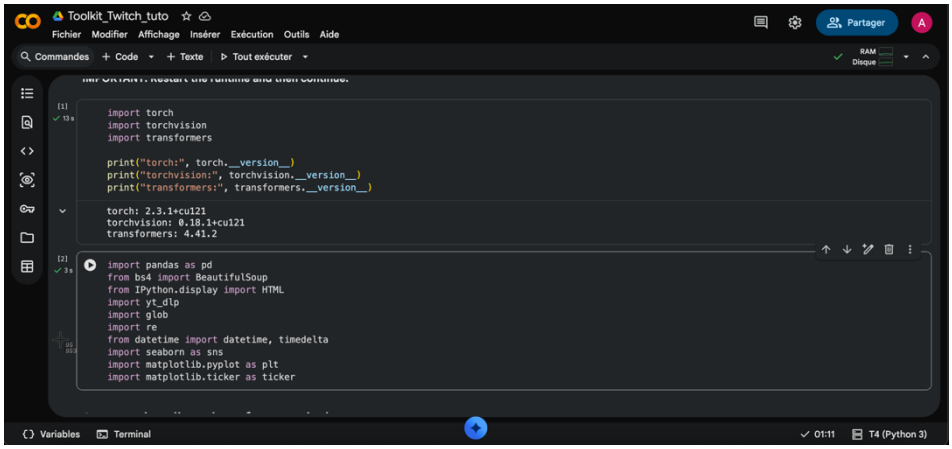
Un voyant vert indique que la cellule a été exécutée, **un voyant rouge** qu'elle n'a pas été exécutée :



ÉTAPE 2 : redémarrer la session. Menu : Exécution > Redémarrer la session :

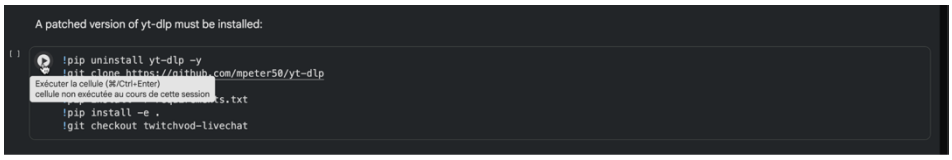


ÉTAPE 3 : exécuter les deux cellules suivantes :

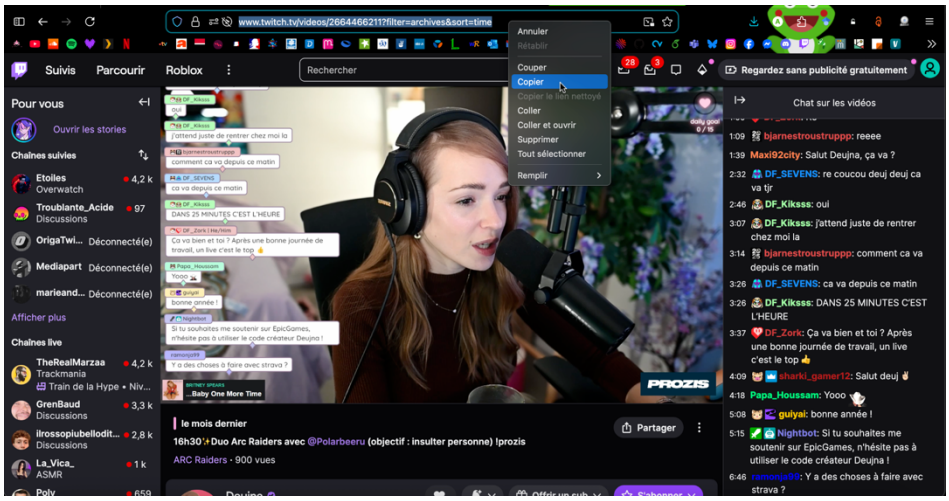


3.2 Téléchargement des données de Twitch

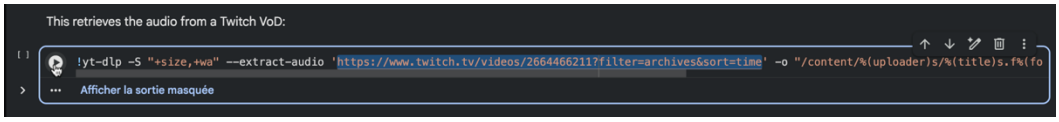
ÉTAPE 4 : installer le logiciel yt-dlp pour télécharger les audio des VODs de Twitch en exécutant la cellule suivante :



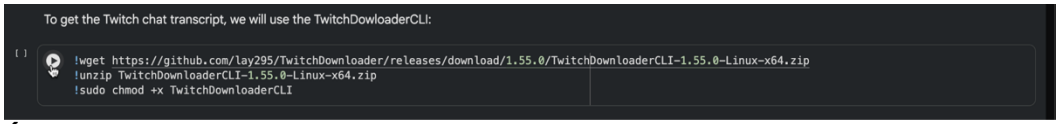
ÉTAPE 5 : aller sur la plateforme Twitch, sélectionner la vidéo à télécharger. Une fois sélectionnée, copier l'URL de la vidéo :



Retourner dans le google colab et coller l'URL dans la cellule suivante (coller l'URL entre les apostrophes, à la place de la partie en surbrillance sur la photo), puis exécuter la cellule [pour obtenir une partie de l'audio, se référer à l'**étape 17 B**] :



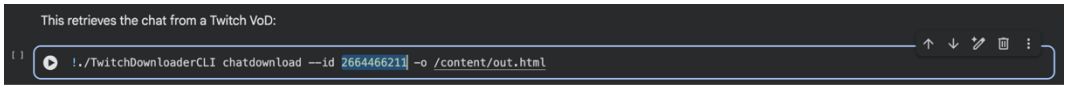
ÉTAPE 6 : installer le TwitchDownloaderCLI pour télécharger les chats des VODs de Twitch en exécutant la cellule suivante :



ÉTAPE 7 : dans l'URL de la VOD Twitch sélectionnée, copier le numéro d'identifiant de la VOD sélectionnée. Il s'agit du numéro en surbrillance sur l'image :



Coller ce numéro dans la cellule suivante à exécuter afin de télécharger le chat de la VOD sélectionnée, puis exécuter la cellule (coller à la suite de "id", la partie en surbrillance sur l'image) [pour obtenir une partie du chat, se référer à l'**étape 17 C**] :



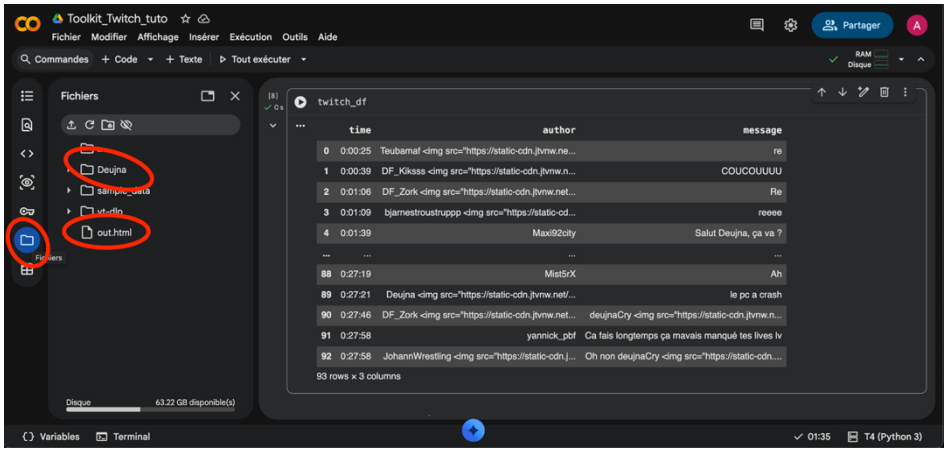
ÉTAPE 8 : exécuter les deux cellules suivante pour (a) structurer les données chat :



(b) vérifier le fichier chat (il doit apparaître tel que sur l'image ci-dessous) :



Le fichier (qui se nomme “out.html”) est disponible au téléchargement dans le dossier à gauche (**attention, pensez à bien le télécharger, le google CoLab ne sauvegarde rien**) :



Note : l’audio est disponible dans le dossier au nom du streamer (dans l’exemple ci-dessus, il s’agit du dossier “Deujna”).

3.3 Transcription des données audio

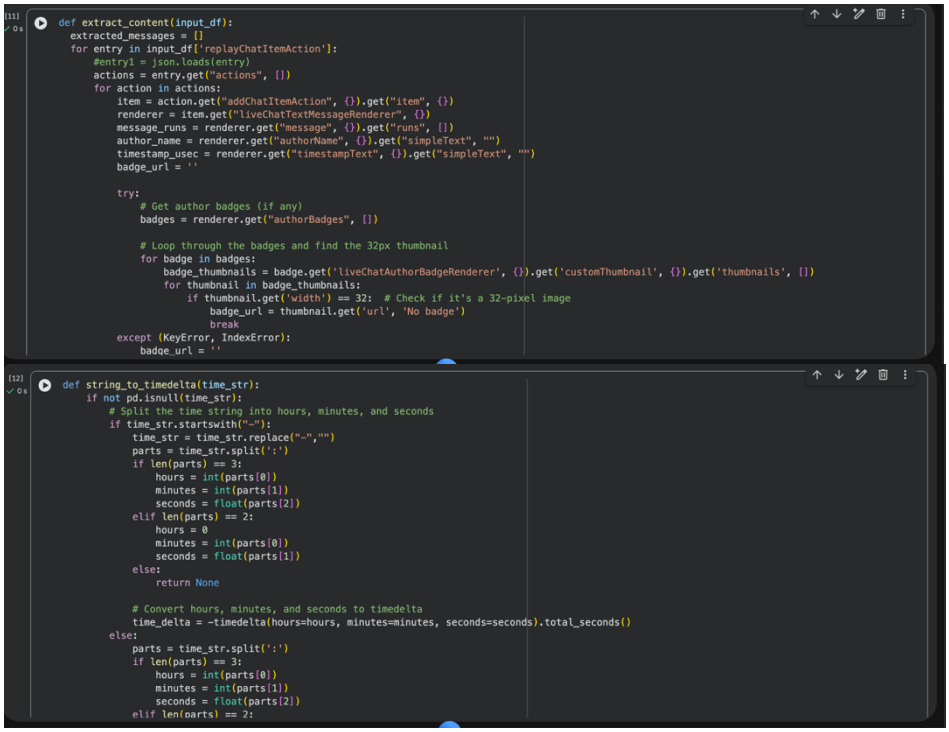
ÉTAPE 9 : exécuter la première cellule de cette section (3) pour transcrire les données audio :



Puis vérifier que la transcription a été correctement exécutée :



ÉTAPE 10 : exécuter les deux prochaines cellules pour structurer les données :



```
def extract_content(input_df):
    extracted_messages = []
    for entry in input_df['replayChatItemAction']:
        #entry1 = json.loads(entry)
        actions = entry.get("actions", [])
        for action in actions:
            item = action.get("addChatItemAction", {}).get("item", {})
            renderer = item.get("liveChatTextMessageRenderer", {})
            message_runs = renderer.get("message", {}).get("runs", [])
            author_name = renderer.get("authorName", {}).get("simpleText", "")
            timestamp_usec = renderer.get("timestampText", {}).get("simpleText", "")
            badge_url = ''

            try:
                # Get author badges (if any)
                badges = renderer.get("authorBadges", [])

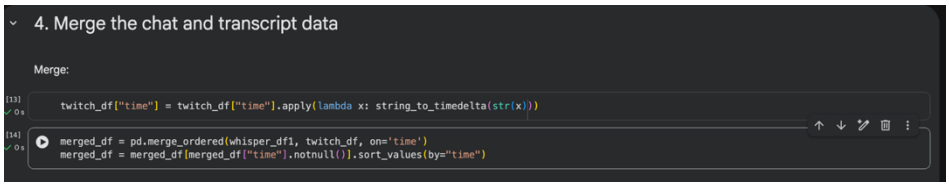
                # Loop through the badges and find the 32px thumbnail
                for badge in badges:
                    badge_thumbnails = badge.get('liveChatAuthorBadgeRenderer', {}).get('customThumbnail', {}).get('thumbnails', [])
                    for thumbnail in badge_thumbnails:
                        if thumbnail.get('width') == 32:  # Check if it's a 32-pixel image
                            badge_url = thumbnail.get('url', 'No badge')
                            break
            except (KeyError, IndexError):
                badge_url = ''
```

```
def string_to_timedelta(time_str):
    if not pd.isnull(time_str):
        # Split the time string into hours, minutes, and seconds
        if time_str.startswith("-"):
            time_str = time_str.replace("-","")
            parts = time_str.split(':')
            if len(parts) == 3:
                hours = int(parts[0])
                minutes = int(parts[1])
                seconds = float(parts[2])
            elif len(parts) == 2:
                hours = 0
                minutes = int(parts[0])
                seconds = float(parts[1])
            else:
                return None

            # Convert hours, minutes, and seconds to timedelta
            time_delta = -timedelta(hours=hours, minutes=minutes, seconds=seconds).total_seconds()
        else:
            parts = time_str.split(':')
            if len(parts) == 3:
                hours = int(parts[0])
                minutes = int(parts[1])
                seconds = float(parts[2])
            elif len(parts) == 2:
```

3.4 Alignement du chat et de la transcription audio

ÉTAPE 11 : fusionner les fichiers chat et la transcription en exécutant les deux prochaines cellules :



```
4. Merge the chat and transcript data

Merge:

twitch_df["time"] = twitch_df["time"].apply(lambda x: string_to_timedelta(str(x)))

merged_df = pd.merge_ordered(whisper_df1, twitch_df, on='time')
merged_df = merged_df[merged_df["time"].notnull()].sort_values(by="time")
```

ÉTAPE 12 : sauvegarder l'alignement au format HTML en exécutant la cellule suivante :

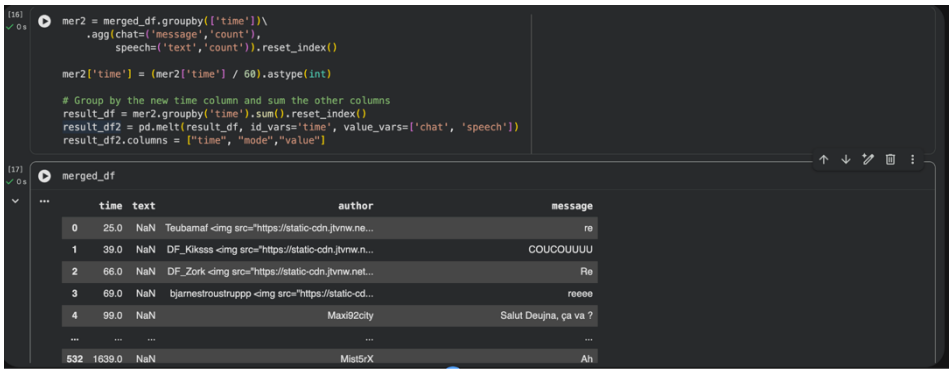


```
Save data to HTML:

# Save DataFrame to HTML
merged_df1 = merged_df.fillna("")
merged_df1.to_html("/content/output.html", index=False, render_links=True, escape=False)

# HTML Content to add (Bootstrap + Custom Styles + JavaScript for sortable table)
html_content = """
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
<style>
  body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
  }
  .table-wrapper {
    padding: 20px;
    max-width: 100%;
    overflow-x: auto; /* Allow horizontal scrolling if necessary */
  }
```

ÉTAPE 13 : terminer l'alignement en exécutant la cellule suivante puis vérifier l'alignement en exécutant la cellule d'après (le fichier doit apparaître comme sur l'image ci-dessous) :



```
mer2 = merged_df.groupby(['time'])\
    .agg(chat=('message','count'),
         speech=('text','count')).reset_index()

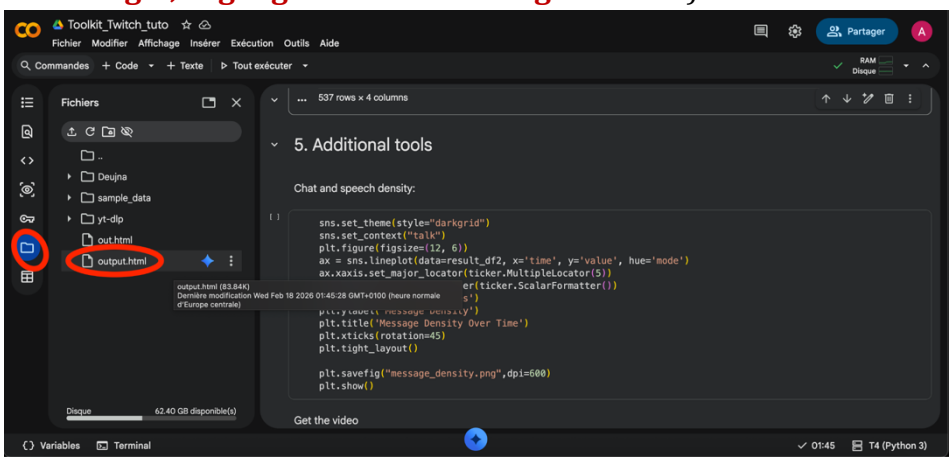
mer2['time'] = (mer2['time'] / 60).astype(int)

# Group by the new time column and sum the other columns
result_df = mer2.groupby('time').sum().reset_index()
result_df2 = pd.melt(result_df, id_vars='time', value_vars=['chat', 'speech'])
result_df2.columns = ["time", "mode","value"]
```

```
merged_df
```

	time	text	author	message
0	25.0	NaN	Teubamaf <img src="https://static-cdn.jtvnw.ne...	re
1	39.0	NaN	DF_Kiksss <img src="https://static-cdn.jtvnw.n...	COUCOUUUU
2	66.0	NaN	DF_Zork <img src="https://static-cdn.jtvnw.net...	Re
3	69.0	NaN	bjarnestroustruppp <img src="https://static-cd...	reeee
4	99.0	NaN	Maxi92city	Salut Deujna, ça va ?
...	...	...	...	...
532	1639.0	NaN	Mist5rX	Ah

À ce stade : les différentes étapes sont terminées pour obtenir les données de Twitch alignées (audio et chat). Le fichier a été nommé "output.html" et est disponible au téléchargement dans le même dossier que le fichier chat (**attention, pensez à bien le télécharger, le google colab ne sauvegarde rien**) :



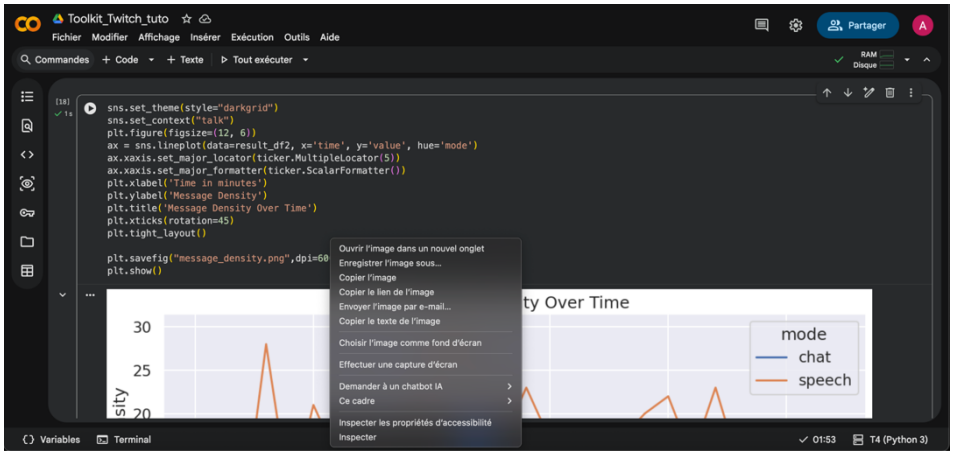
Le fichier HTML est un fichier en 4 colonnes : (i) le temps ; (ii) la transcription de l'audio ; (iii) les pseudonymes des utilisateur·rices ; (iv) les messages dans le chat. Les messages et la transcription sont donc alignés selon le temps. Voici un exemple de l'output que vous devriez obtenir :

time	text	author	message
25.000		Teubamaf	re
39.000		DF_Kiksss	COUCOUUUU
66.000		DF_Zork	Re
69.000		bjarnestroustruppp	reeee
99.000		Maxi92city	Salut Deujna, ça va ?
152.000		DF_SEVENS	re coucou deuj deuj ca va tjr
157.624	Allô ?		
159.504	Vous êtes fatigué ou quoi la team ?		
162.605	Vous êtes plutôt fatiguée ou pas ?		
166.000		DF_Kiksss	oui
167.250	Comment ça va ?		
168.271	Hello hello !		

3.5 Quelques outils additionnels

Voici quelques **outils additionnels** pour compléter les données (voir les cellules sous la partie 5) :

ÉTAPE 14 : pour **obtenir un graphique permettant la visualisation de la densité du chat et des tours de parole**, il suffit d'exécuter la cellule suivante :



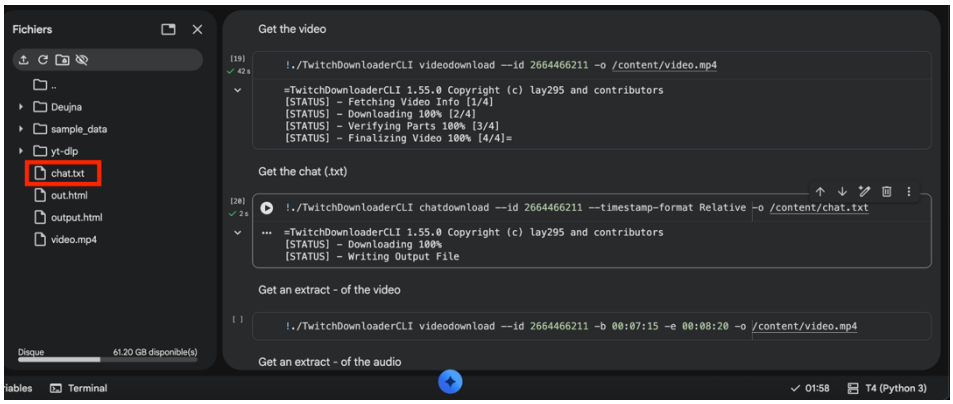
Vous pouvez sauvegarder l'image .png en faisant un clic droit dessus puis "enregistrer l'image sous".

ÉTAPE 15 : pour **obtenir la vidéo au format .mp4**, copier l'identifiant de la VOD disponible dans l'URL et le coller à la suite de "id" (comme en **étape 7**) dans la cellule suivante, puis exécuter la cellule [pour obtenir une partie de la vidéo, se référer à l'**étape** 17 A] :



La vidéo est disponible dans le dossier et se nomme "video.mp4" (attention, pensez à bien la télécharger, le google colab ne sauvegarde rien).

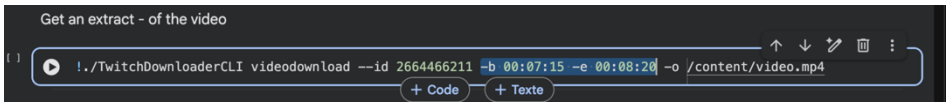
ÉTAPE 16 : pour **obtenir le chat au format .txt**, copier l'identifiant de la VOD disponible dans l'URL et le coller à la suite de "id" (comme en **étape 7**) dans la cellule suivante, puis exécuter la cellule :



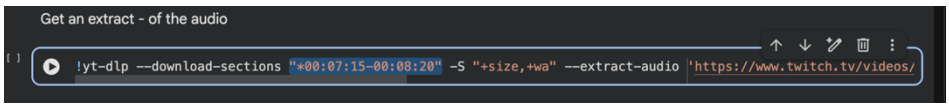
Le chat est disponible dans le dossier et se nomme “chat.txt” (attention, pensez à bien la télécharger, le google colab ne sauvegarde rien).

ÉTAPE 17 : Pour **ne télécharger qu’une partie de la VOD (chat, vidéo et audio)**, vous pouvez utiliser les trois cellules suivantes en remplacement de certaines étapes précédentes :

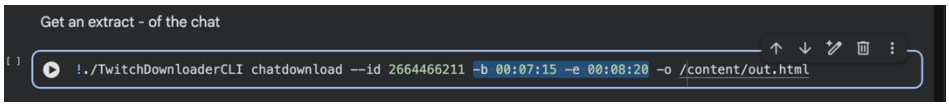
- **A** [en remplacement de l’**étape 15**] : pour **obtenir une partie de la vidéo**, remplacer les temps dans la cellule suivante (par exemple, dans l’image ci-dessous, les parties en surbrillance correspondent à une sélection de la VOD entre 7 minutes 15 secondes et 8 minutes 20 secondes) puis exécuter la cellule :



- **B** [en remplacement de l’**étape 5**] : pour **obtenir une partie de l’audio**, remplacer les temps dans la cellule suivante (par exemple, dans l’image ci-dessous, les parties en surbrillance correspondent à une sélection de la VOD entre 7 minutes 15 secondes et 8 minutes 20 secondes), puis exécuter la cellule :

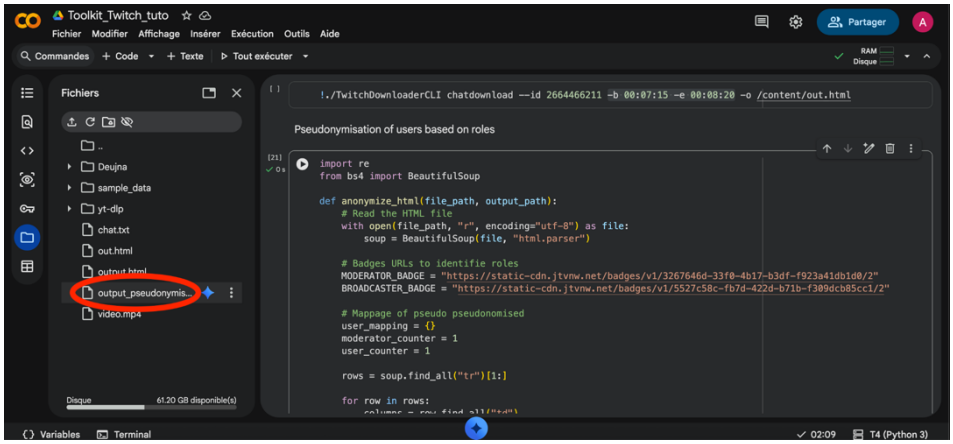


- **C** [en remplacement de l’**étape 7**] : pour **obtenir une partie du chat**, remplacer les temps dans la cellule suivante (par exemple, dans l’image ci-dessous, les parties en surbrillance correspondent à une sélection de la VOD entre 7 minutes 15 secondes et 8 minutes 20 secondes), puis exécuter la cellule :



Important : pour les extraits, il est crucial de choisir **exactement les mêmes extraits (pour la vidéo, l’audio et le chat)** et de bien suivre l’ordre d’exécution des cellules et donc des étapes (ainsi : une fois l’étape 7 remplacée par la 17 C, continuez avec l’étape 18).

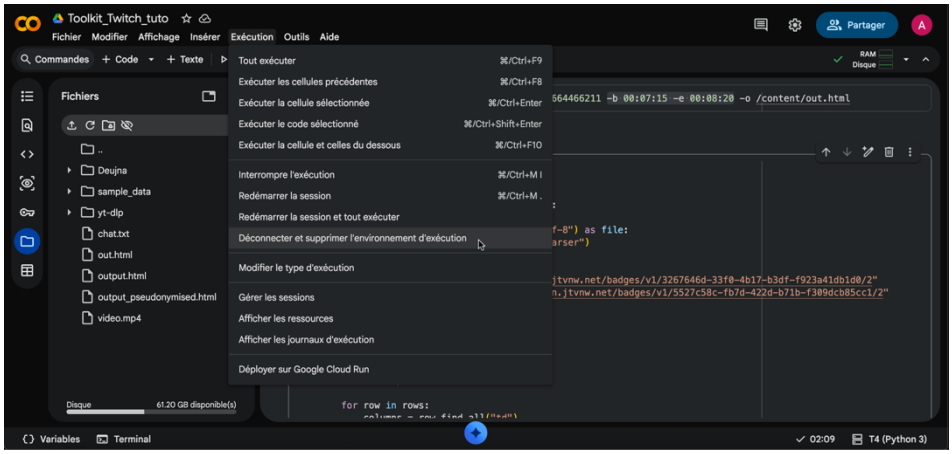
Pour obtenir une pseudonymisation des fichiers alignés, exécuter la dernière cellule :



Le fichier pseudonymisé est disponible dans le dossier et se nomme “output_pseudonymisé.html” (**attention, pensez à bien la télécharger, le google colab ne sauvegarde rien**). Il apparaît alors comme ceci :

time	text	author	message
10.000		User1	bgrLOUTRE Bonjour avamindLuv
12.000		User2	avamindBONJ avamindLOVE
22.340	Oh non, pas un arc, hein.		
24.000		User3	avamindLuv avamindLuv avamindLuv avamindLuv avamindLuv
29.000		User4	avamindBONJ avamindBONJ avamindLuv avamindLuv
30.572	Qu'est-ce que... Oh non, hein.		
32.135	Ah, faut arrêter avec les arcs, hein.		
33.000		User5	avamindBONJ acastC1 acastAcastC2 acastC3 avamindLuv
33.737	C'est fini, ça.		
40.000		User6	bonjour avamindLuv

Important : pour télécharger, transcrire et aligner de nouvelles données de Twitch, il est important de **nettoyer l’environnement avant de revenir à la première étape**. Pour cela, dans Menu : Exécution > Déconnecter et supprimer l’environnement d’exécution :



4 Téléchargement des données Twitter/X à l'aide des freewares – S. Jeon

Lorsque les données multimédia sont disponibles sous forme de fichier vidéo brut, il convient d'utiliser les logiciels libres. Dans cette sous-section, trois logiciels libres sont présentés. Ils ont été sélectionnés pour leur compatibilité avec les trois systèmes d'exploitation (Window, Mac, Linux).

- Logiciel d'édition : VideoProc Vlogger https://www.videoproc.com/video-editing-software/guide-free-mac/cut-split.htm?ttsoft=vpvmac-1.4-tips-trial
- Outil de transcription automatique du fichier audio : Speechmatics https://www.speechmatics.com

Concernant les posts sur X, nous procédons par l'enregistrement d'écran afin d'inclure les données multimédia intégrantes.

- Outil d'enregistrement de l'écran : OBS studio https://obsproject.com/fr

Il s'agit d'un tutoriel prêt-à-l'emploi et adapté aux besoins du projet, dans la mesure où nous nous concentrerons plus spécifiquement sur les fonctionnalités nécessaires au traitement des données multimédia, à savoir :

- Éditer une vidéo pour en extraire des segments,
- Transcrire automatiquement ces segments en vue de les inclure dans le corpus
- Enregistrer l'écran pour archiver et recueillir des posts Twitter/X.

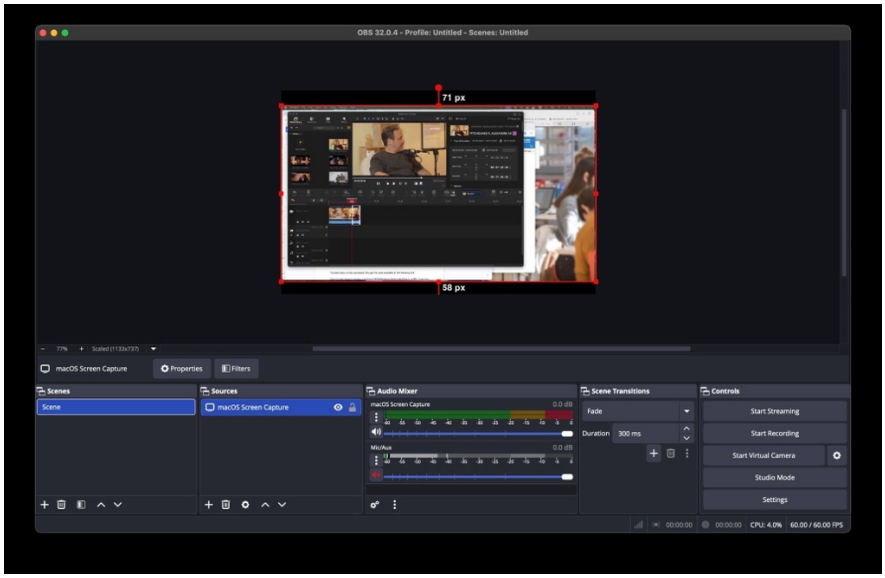
Vous pouvez retrouver ce tutoriel en vidéo sur YouTube au lien suivant :
https://youtu.be/1GRFJ8KwxKQ

4.1 Éditer une vidéo avec VideoProc Vlogger

Installez le logiciel.

4.2 Enregistrer l'écran avec OBS studio

Installez le logiciel.
Une fois le OBS installé, lancez le logiciel. L'interface principale apparaît comme illustrée ci-dessous.



Il convient de définir précisément la zone de l'écran à enregistrer, de manière à ce que l'intégralité du contenu du post X soit clairement visible.
Pour cela, **cliquez sur l'aperçu de l'écran affiché** dans la fenêtre principale de l'interface. Assurez-vous que le petit écran sélectionné est encadré en rouge, puis cliquez sur le **bouton Properties**.
Trois options sont proposées : Display Capture, Window Capture, Application Capture. Vous pouvez par exemple sélectionner l'option : Window Capture, puis sélectionner le navigateur web dans lequel le post ciblé est affiché. Il est également possible de rendre invisible le curseur en décochant "Show cursor".
Dans la **fenêtre Audio Mixer** : une fois la zone de capture définie, assurez-vous que le son du système (macOS Screen Capture ou Desktop Audio) est activé et que le microphone est désactivé.
Dans la **fenêtre Controls** : avant de procéder à l'enregistrement, vérifiez le format du fichier d'output. Cliquez sur le bouton Settings. Dans l'onglet Output, allez dans la section Recording, puis sélectionnez dans Recording Format : Hybrid MP4 (.mp4). Cliquez sur le bouton OK pour valider.

À ce stade, tous les paramètres attendus sont configurés, c'est-à-dire la zone de capture définie, le format de l'output en MP4, le micro désactivé, le son du système activé.

Lancer l'enregistrement
Dans la fenêtre du navigateur où le post X est affiché, cliquez sur le bouton lecture.
Dans la **fenêtre Controls** : cliquez sur Start recording. Pendant l'enregistrement le bouton devient bleu avec Stop recording affiché. Lorsque la vidéo du post s'achève, cliquez sur sur le bouton Stop Recording. Vérifiez ensuite l'emplacement du fichier enregistré, qui s'affiche en bas de l'interface.

4.3 Transcrire automatiquement un fichier audio avec Speechmatics

Rendez-vous sur le site Speechmatics. Créez un compte utilisateur. Générez une API key, qui vous sera demandée.

Dans le panneau à gauche de l'interface, cliquez sur Speech to text > File transcription. La fenêtre Create s'affiche. Sélectionnez l'option Batch Transcribe previously recorded media, puis cliquez sur le bouton Upload a file. Cette fonctionnalité est compatible avec les formats suivants : aac, amr, flac, m4a, mp3, mp4, mpeg, ogg, wav, pour des fichiers d'une taille inférieure à 1Go.

Extraction des fichiers audio (convert_media_audio.py)
Afin de réduire le temps de traitement, il est recommandé d'utiliser un format audio compact comme mp3. Notez toutefois que les fichiers WAV sont requis pour l'analyse phonologique sous Praat. Dans le cadre du projet, les fichiers vidéo (.mp4) et audio (.wav)

sont associés au fichier .eaf (ELAN). Cela implique que les fichiers vidéos doivent être convertis en WAV.
Le script *convert_media_audio.py* permet cette conversion. Voici ci-dessous son mode d'emploi :

- (Installez python.)
- Sélectionnez un dossier de travail où vous déposerez le script *convert_media_audio.py* ; ainsi que les fichiers média à convertir
- Ouvrez le Terminal.
- Précisez le chemin d'accès pour accéder au dossier de travail à l'aide de la commande : cd /emplacement/du/dossier/de/votre/choix
- Lancez le script en saisissant : python convert_media_audio.py
- Suivez les instructions qui s'afficheront dans le Terminal.

Gestion des formats dans la chaîne de traitement : chaque donnée est déclinée en plusieurs formats avec des fonctions différentes. Voici les formats principaux :

- mp4-> archivage et association au fichier .eaf
- wav-> association au fichier .eaf et analyse prosodique (Praat)
- mp3-> transcription automatique
- eaf-> alignement temporel, segmentation en unité illocutionnaire
- txt-> transcriptions

4.4 Transcrire automatiquement un fichier audio avec asr_batch.py

Vous pouvez également utiliser le script asr_batch.py pour générer une transcription automatique de fichiers audio (automatic speech recognition). Les instructions d'installation de Library *faster-whisper* et d'exécution sont intégrées en tête du script.

(step1) In Terminal

- Install faster-whisper
  ```
  pip3 install faster-whisper
  ```
- After installation, verify:
  ```
  python3 -c "from faster_whisper import WhisperModel; print('OK')"
  ```

(step2)

- In the same directory where asr_batch.py is saved, create a folder entitled "audio_in" then place your input audio files inside

(step3) Run transcription in Terminal:

- Italian speech recognition

```
python3 asr_batch.py ./audio_in ./out --engine whisper --model medium --language it --formats txt,json
```

- French speech recognition

```
python3 asr_batch.py ./audio_in ./out --engine whisper --model medium --language fr --formats txt,json
```